

API Contracts

Project Name

CarbonWise

Version: 1.0

API Style: REST

Backend Framework: NestJS

Authentication: JWT

Validation: Zod

Status: Approved

1. Purpose

This document defines all API contracts for CarbonWise.

The API layer is responsible for:

- Authentication
- Carbon Calculations
- Recommendations
- Goal Management
- Carbon Twin Simulations
- Analytics

The design prioritizes:

- Code Quality
 - Security
 - Efficiency
 - Maintainability
-

2. API Standards

Base URL

Development

```
http://localhost:3001/api/v1
```

Production

```
https://api.carbonwise.com/api/v1
```

Content Type

```
Content-Type: application/json
```

Request Format

```
{  
  "data": {}  
}
```

Success Response Format

```
{  
  "success": true,  
  "message": "Request successful",  
  "data": {}  
}
```

Error Response Format

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request"
  }
}
```

3. Authentication

All protected routes require:

```
Authorization: Bearer <access_token>
```

Token Strategy

Access Token

15 Minutes

Refresh Token

30 Days

4. Authentication APIs

Register User

Endpoint

POST /auth/register

Request

```
{
  "fullName": "Rahul Sharma",
  "email": "rahul@example.com",
  "password": "StrongPassword123"
}
```

Validation Rules

Name Required

Valid Email

Password Minimum 8 Characters

Response

```
{
  "success": true,
  "message": "User registered successfully"
}
```

Login User

Endpoint

```
POST /auth/login
```

Request

```
{
  "email": "rahul@example.com",
  "password": "StrongPassword123"
}
```

Response

```
{
  "success": true,
  "data": {
    "accessToken": "...",
    "refreshToken": "..."
  }
}
```

Refresh Token

Endpoint

```
POST /auth/refresh
```

Request

```
{  
  "refreshToken": "..."  
}
```

Response

```
{  
  "accessToken": "..."  
}
```

Logout

Endpoint

```
POST /auth/logout
```

Response

```
{  
  "success": true  
}
```

5. User APIs

Get Profile

Endpoint

```
GET /users/profile
```

Response

```
{
  "id": "uuid",
  "fullName": "Rahul Sharma",
  "email": "rahul@example.com"
}
```

Update Profile

Endpoint

```
PATCH /users/profile
```

Request

```
{
  "fullName": "Rahul Sharma",
  "dietType": "Vegetarian"
}
```

Response

```
{
  "success": true
}
```

6. Carbon Calculator APIs

These APIs directly support:

Understand

from the challenge statement.

Calculate Carbon Footprint

Endpoint

`POST /carbon/calculate`

Request

```
{
  "transportation": {
    "carKm": 120,
    "busKm": 20
  },
  "food": {
    "meatMeals": 5
  },
  "energy": {
    "electricityKwh": 250
  },
  "shopping": {
    "clothingItems": 3
  }
}
```

Response

```
{
  "success": true,
  "data": {
    "carbonScore": 68,
    "totalEmission": 154.6,
    "unit": "kgCO2e"
  }
}
```

Save Activity

Endpoint

```
POST /activities
```

Request

```
{
  "category": "TRANSPORTATION",
  "activityName": "Car Travel",
  "quantity": 120,
  "unit": "km"
}
```

Response

```
{
  "success": true,
  "activityId": "uuid"
}
```

Get Activities

Endpoint

```
GET /activities?page=1&limit=20
```

Response

```
{
  "page": 1,
  "limit": 20,
  "total": 120,
  "data": []
}
```

7. Dashboard APIs

These APIs directly support:

Track

Dashboard Summary

Endpoint

```
GET /dashboard
```

Response

```
{
  "carbonScore": 68,
  "monthlyEmission": 180,
}
```

```
"activeGoals": 2,  
"recommendations": 4  
}
```

Dashboard Analytics

Endpoint

```
GET /dashboard/analytics
```

Response

```
{  
  "monthlyTrend": [],  
  "categoryBreakdown": []  
}
```

8. Recommendation APIs

These APIs directly support:

```
Reduce
```

Generate Recommendations

Endpoint

```
POST /recommendations/generate
```

Response

```
{
  "success": true,
  "data": [
    {
      "title": "Use Public Transport",
      "priority": "HIGH",
      "estimatedSavings": 120
    }
  ]
}
```

Get Recommendations

Endpoint

```
GET /recommendations
```

Query

```
?page=1&limit=10
```

Response

```
{
  "data": []
}
```

Accept Recommendation

Endpoint

```
POST /recommendations/:id/accept
```

Response

```
{  
  "success": true  
}
```

Complete Recommendation

Endpoint

```
POST /recommendations/:id/complete
```

Response

```
{  
  "success": true  
}
```

9. Goal APIs

Create Goal

Endpoint

```
POST /goals
```

Request

```
{
  "title": "Reduce Transportation Emissions",
  "targetReduction": 20,
  "targetDate": "2027-01-01"
}
```

Response

```
{
  "goalId": "uuid"
}
```

Get Goals

Endpoint

```
GET /goals
```

Response

```
{
  "data": []
}
```

Update Goal

Endpoint

```
PATCH /goals/:goalId
```

Response

```
{  
  "success": true  
}
```

Delete Goal

Endpoint

```
DELETE /goals/:goalId
```

Response

```
{  
  "success": true  
}
```

10. Carbon Twin APIs

This is the platform's differentiating feature.

Create Simulation

Endpoint

```
POST /carbon-twin/simulate
```

Request

```
{
  "scenario": {
    "transportation": "public_transport",
    "meatReduction": 3
  }
}
```

Response

```
{
  "currentEmission": 250,
  "projectedEmission": 180,
  "potentialSavings": 70
}
```

Get Previous Simulations

Endpoint

```
GET /carbon-twin/history
```

Response

```
{  
  "data": []  
}
```

11. Notifications API

Get Notifications

Endpoint

```
GET /notifications
```

Mark Read

Endpoint

```
PATCH /notifications/:id/read
```

12. Validation Standards

All requests validated using:

```
Zod
```

Example Schema

```
const CreateGoalSchema = z.object({  
  title: z.string().min(3),  
  targetReduction: z.number().positive()  
});
```

13. Security Controls

Supports:

Security (Medium Impact)

Helmet

Enabled globally.

Rate Limiting

100 Requests / Minute

per user.

Input Validation

All endpoints validated.

JWT Authentication

Protected routes require authentication.

Password Security

bcrypt

used for hashing.

CORS

Restricted origins only.

14. Efficiency Standards

Supports:

Efficiency (Medium Impact)

Pagination

Mandatory on:

- Activities
 - Recommendations
 - Notifications
-

Redis Caching

Applied to:

- Dashboard
 - Analytics
 - Recommendations
-

Query Optimization

Use:

Indexed Queries

Avoid:

SELECT *

15. Error Codes

Code	Meaning
VALIDATION_ERROR	Invalid Request
UNAUTHORIZED	Authentication Failed
FORBIDDEN	Access Denied
NOT_FOUND	Resource Missing
RATE_LIMITED	Too Many Requests
INTERNAL_ERROR	Server Error

16. API Testing Strategy

Supports:

Testing (Low Impact)

Unit Testing

Tool:

Jest

Integration Testing

Tool:

Supertest

E2E Testing

Tool:

Playwright

Critical Flows

Must be tested:

Register

Login

Carbon Calculation

Recommendations

Goals

Carbon Twin

17. Success Criteria

The API layer is successful when:

- ✓ Authentication is secure
- ✓ Carbon calculations are accurate
- ✓ Recommendations are personalized
- ✓ Carbon Twin simulations work reliably
- ✓ APIs remain performant
- ✓ Validation prevents malformed requests
- ✓ Test coverage goals are achieved
- ✓ The API fully supports:
 - Understand
 - Track
 - Reduce

requirements of the challenge.